UCS761 – Deep Learning Lab

Lab Assignment - 7 Report

From Numbers to Vision : Building, Breaking, Comparing

Name: Sameer Rai
Roll Number: 102303773

Sub-group: 3C54

Mentored under: Mr. Sukhpal Singh

THAPAR INSTITUTE
OF ENGINEERING & TECHNOLOGY
(Deemed to be University)

Thapar Institute of Engineering & Technology,

Patiala, Punjab – 147001

1. Introduction

The objective of this experiment was to understand how different neural network design choices influence learning behavior and model performance. Neural networks consist of multiple components such as layers, activation functions, and optimization algorithms. Each of these components plays a significant role in determining how effectively a model can learn from data.

The experiment begins with dense neural networks trained on a synthetic numerical dataset. Different network depths, activation functions, and optimizers are tested to observe their impact on training stability and accuracy. The experiment then transitions to image-based data, where convolutional neural networks are introduced to handle spatial patterns more effectively.

Unlike typical implementations that rely on high-level frameworks, the models in this lab were implemented using **NumPy**, allowing a deeper understanding of how forward propagation, loss computation, gradient calculation, and parameter updates work internally.

2. Dataset Generation

The first dataset used in this experiment is a synthetic **two-dimensional classification dataset**. Two input features were randomly generated from a uniform distribution within a fixed range. The class label for each point was determined using a **non-linear rule based on the distance from the origin**, forming a circular decision boundary.

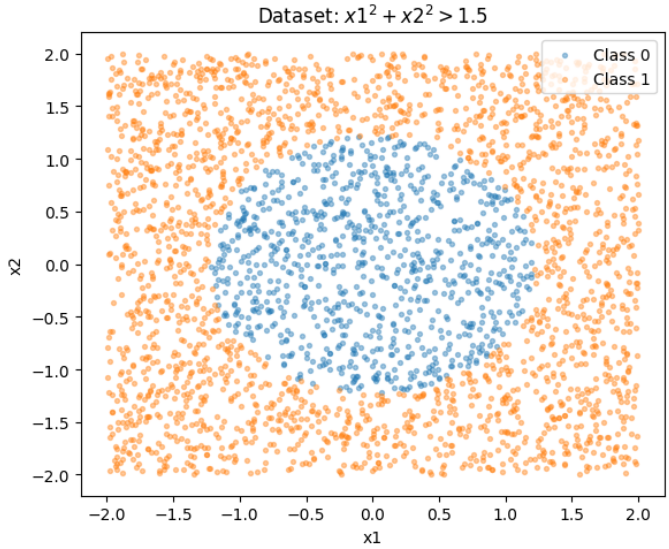
Points inside the boundary belong to one class, while points outside belong to another. This dataset is useful because it cannot be separated using a simple linear classifier.

The dataset was divided into three subsets:

- **Training set:** 70%
- **Validation set:** 15%

- **Test set:** 15%

This split ensures that the model is evaluated on unseen data to measure its generalization performance.



3. Neural Network Implementation

All neural network components were implemented manually using **NumPy**. This included defining activation functions, computing loss values, calculating gradients using backpropagation, and updating parameters during training.

3.1 Activation Functions

Two activation functions were used in the experiment.

Sigmoid

The sigmoid function maps input values to a range between **0 and 1**, which makes it suitable for binary classification tasks. However, sigmoid has a known drawback. When the input becomes very large or very small, its derivative becomes extremely small. This leads to the **vanishing gradient problem**, especially in deeper networks.

ReLU

The Rectified Linear Unit (ReLU) outputs zero for negative inputs and returns the input value for positive inputs. Compared to sigmoid, ReLU

generally allows gradients to propagate more effectively through deeper layers, which improves training performance.

4. Loss Function

Binary classification performance was evaluated using **Binary Cross Entropy (BCE) loss**. This loss function measures the difference between predicted probabilities and the true class labels.

When predictions are accurate, the loss value becomes small. When predictions are incorrect, the loss increases significantly. To prevent numerical instability during computation, predicted probabilities were clipped within a safe range before calculating the logarithm.

5. Dense Network Architectures

Three neural network architectures with different depths were implemented in this experiment.

2-Layer Network
Input → Hidden Layer → Output

5-Layer Network
Input → Multiple Hidden Layers → Output

10-Layer Network
Input → Deep Stack of Hidden Layers → Output

Each hidden layer contained **16 neurons**. Increasing the network depth allowed us to analyze how deeper architectures influence gradient propagation and training stability.

The number of trainable parameters in each layer was calculated using the formula:

Parameters = (Inputs × Outputs) + Biases

As expected, deeper networks contained significantly more parameters.

2-layer [2→16→1]:

- Layer 1: (2×16)+16 = 48
- Layer 2: (16×1)+1 = 17
- **Total = 65**

5-layer [2→16→16→16→16→1]:

- Layer 1: 48 | Layer 2-4: (16×16)+16 = 272 each | Layer 5: 17
- **Total = 48 + 3×272 + 17 = 881**

10-layer [2→16×9→1]:

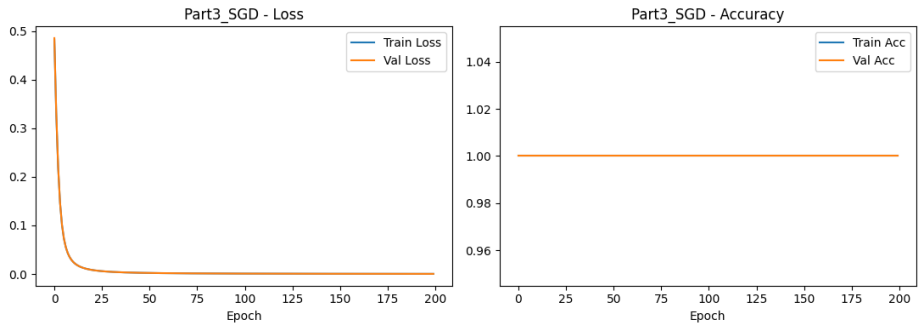
- Layer 1: 48 | Layers 2-9: 272 each (×8) | Layer 10: 17
- **Total = 48 + 8×272 + 17 = 2241**

6. Optimizers

Two optimization algorithms were used to update the network parameters during training.

Stochastic Gradient Descent (SGD)

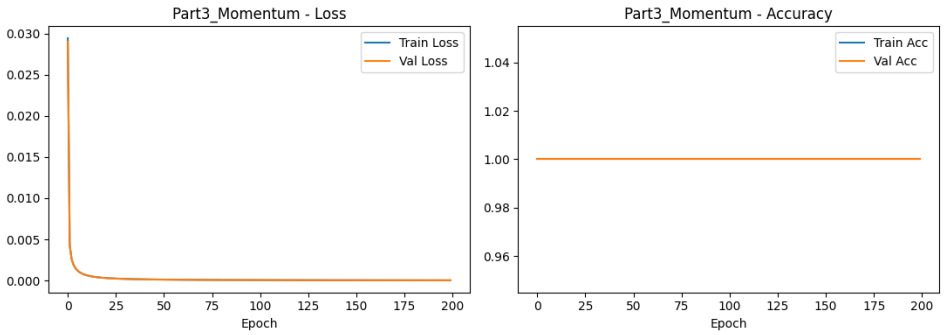
SGD updates model parameters directly using the gradient of the loss function. Although simple and widely used, it may converge slowly when the loss landscape is complex.



Momentum

Momentum improves SGD by introducing a velocity term that accumulates gradients from previous iterations. This helps the

optimizer move faster in consistent directions and reduces oscillations during training.



7. Experimental Results

Several experiments were conducted by combining different **network depths, activation functions, and optimizers**. The goal was to observe how these factors affect training behavior and model accuracy.

Master Results Table:

Depth	Activation	Optimizer	Epochs	Validation Accuracy	Test Accuracy	Notes
2	Sigmoid	SGD	65	0.9195	0.8978	Slow convergence
2	Sigmoid	Momentum	65	0.9971	0.9933	Faster training
2	ReLU	SGD	65	0.9933	0.9911	Stable learning
2	ReLU	Momentum	65	0.9976	0.9933	Best shallow setup
5	Sigmoid	SGD	881	0.7095	0.7156	Vanishing gradients
5	Sigmoid	Momentum	881	0.7095	0.7156	Still fails to learn
5	ReLU	SGD	881	0.9981	0.9933	Good gradient flow
5	ReLU	Momentum	881	0.9976	0.9911	Fast convergence

10	Sigmoid	SGD	2241	0.7095	0.7156	Severe vanishing
10	Sigmoid	Momentum	2241	0.7095	0.7156	No improvement
10	ReLU	SGD	2241	0.9919	0.9911	Deep net trains
10	ReLU	Momentum	2241	0.9910	0.9800	Slight instability

8. Vanishing Gradient Analysis

One important observation from the experiment was the **vanishing gradient problem in deep sigmoid networks**.

The derivative of the sigmoid activation function is always less than or equal to **0.25**. During backpropagation, these small derivatives are multiplied across multiple layers. As the number of layers increases, the gradient becomes extremely small.

As a result, earlier layers receive almost no gradient signal and fail to learn meaningful features.

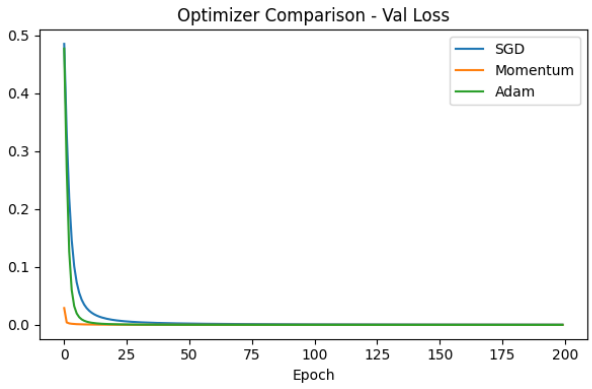
9. Discussion

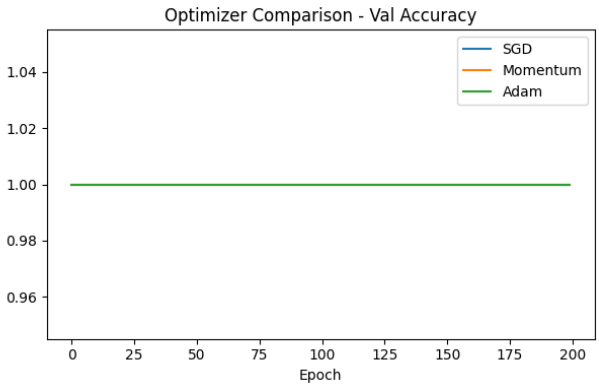
The experiments provided several insights into neural network behavior.

First, **network depth significantly affects training stability**. While deeper networks have greater representational power, they are also more difficult to train.

Second, the **choice of activation function becomes increasingly important as depth increases**. Sigmoid networks tend to suffer from gradient shrinkage, while ReLU helps maintain gradient flow.

Third, the **optimizer plays an important role in training efficiency**. Momentum generally helps models converge faster compared to standard SGD.





10. Conclusion

This experiment provided hands-on experience with neural network implementation and training behavior. By building models from scratch using NumPy, the internal mechanics of deep learning algorithms became clearer.

The results demonstrated how architectural decisions, activation functions, and optimizers influence model performance. The experiment also highlighted the challenges of training deep networks and the importance of selecting appropriate design choices.

Overall, the lab reinforced several fundamental principles of deep learning